Markdown 在线编辑器面试问答

适用于 HR / 技术面试官未看源码，仅基于项目介绍进行提问的场景

使用建议

先把前 6 个重点问题讲顺，再逐步补充技术细节。回答时优先讲业务链路：用户做了什么、组件如何响应、状态如何变化、数据最终保存到哪里。

优先背诵的 6 个重点

- 项目整体介绍
- 为什么用 Pinia + localStorage
- 文档元数据和正文为什么分离
- 自动保存和切换文档怎么保证不丢内容
- 本地 Markdown 文件导入怎么实现
- 导出功能整体怎么实现

一、模拟提问与参考答案

1. 你简单介绍一下这个项目。

这是一个基于 Vue3、Vite、Pinia 和 Vditor 的在线 Markdown 编辑器，主要解决用户在浏览器里编辑和管理 Markdown 文档的问题。它支持实时编辑预览、多文档本地管理、自动保存、本地 .md 文件导入，以及 PNG、PDF、Word、PPT 预览等多格式导出。项目整体是纯前端实现，数据主要保存在浏览器本地，所以刷新页面后也可以恢复上次编辑状态。

2. 你为什么做这个项目？

我想做一个更接近本地文档工作台的 Markdown 工具，不只是能写 Markdown，还能管理多篇文档，支持导入导出，并且不依赖后端也能保存数据。这个项目也练习了 Vue3 项目架构、状态管理、浏览器本地存储、第三方编辑器接入以及浏览器端文件处理。

3. 你在这个项目里主要负责什么？

我主要负责项目整体功能实现和前端架构设计，包括接入 Vditor、用 Pinia 管理文档状态、设计 localStorage 本地持久化方案、实现多文档新建/切换/重命名/删除、自动保存、刷新恢复、本地 Markdown 导入、多格式导出以及移动端侧边栏和异常提示优化。

4. 这个项目最大的亮点是什么？

最大的亮点是把普通 Markdown 编辑器扩展成纯前端的本地文档工作台。它不是只接入一个编辑器，而是围绕编辑器做了多文档管理、本地持久化、文档元数据和正文分离、输入防抖自动保存、切换文档前保存，以及浏览器端导入导出。

5. 这个项目有没有后端？数据存在哪里？

目前没有后端，属于纯前端项目。文档数据保存在浏览器 localStorage 里。文档列表、当前文档 ID 和每篇文档正文是分开存储的，这样刷新页面后可以恢复上次编辑状态，也能做到不上传用户内容。

6. 这个项目整体架构是怎样的？

可以分成组件层、状态层、存储层和编辑器层。组件层包括 Main、Sidebar、HeaderNav 和导出页；状态层是 Pinia 的 docStore；存储层是 LocalStorageAdapter；编辑器层是 Vditor。用户操作组件，组件调用 Pinia action，Pinia 调用存储适配器，适配器读写 localStorage。

7. 为什么选择 Pinia？

因为文档状态会被多个组件共享。侧边栏需要文档列表，主编辑器需要当前正文，导出页要读取当前文档内容，顶部导入功能也要创建和切换文档。Pinia 可以作为统一状态源，集中管理文档列表、当前文档 ID 和正文内容。

8. 为什么封装 localStorage，而不是直接在组件里用？

主要是为了降低耦合。如果组件里到处直接操作 localStorage，后续想换成 IndexedDB 或云端接口会很麻烦。封装成存储适配层后，组件和 Pinia 只关心保存、读取、新建、删除等方法，不关心底层怎么存。

9. 文档数据为什么要拆成元数据和正文？

因为侧边栏只需要展示文档标题、ID、更新时间等元数据，不需要加载所有正文。如果所有正文都塞到一个 JSON 数组里，每次渲染侧边栏都要读写整个大对象。分开存以后，列表加载更轻，正文按文档 ID 单独读取和保存。

10. 用户刷新页面后，怎么恢复上次编辑状态？

项目会在 localStorage 里保存当前文档 ID。页面初始化时读取文档列表和上次激活的文档 ID，再根据这个 ID 读取对应正文，最后把内容设置到 Vditor 编辑器里。

11. 自动保存怎么实现？

Vditor 提供输入回调，用户输入时会触发这个回调。我没有每次输入都立即写 localStorage，而是做了 1 秒防抖。用户停止输入 1 秒后，通过 Vditor 的 getValue 获取当前 Markdown 内容，然后调用 Pinia 的保存方法，最终写入 localStorage。

12. 为什么自动保存要做防抖？

因为 localStorage 是同步 API，如果每输入一个字符就写一次，会频繁阻塞主线程，尤其是文档比较长时会有性能浪费。防抖可以把连续输入合并成一次保存，既保证内容较及时保存，又减少写入次数。

13. 切换文档时，怎么保证当前内容不丢？

切换文档前会先做一次立即保存。用户可能刚输入完，防抖保存还没触发，如果这时直接切换，当前内容可能没写入本地。正确流程是先保存当前 Vditor 内容，再切换当前文档 ID，再读取新文档正文，最后 setValue 到 Vditor。

14. 多文档切换大概是怎么流转的？

侧边栏负责展示文档列表。用户点击某篇文档后，侧边栏通知主编辑页面。主编辑页面先保存当前编辑器内容，再通过 Pinia 切换当前文档，读取新文档正文，最后调用 Vditor 的 setValue，把新内容显示到编辑器里。

15. Vditor 在项目里承担什么角色？

Vditor 是 Markdown 编辑器内核，负责 Markdown 编辑、实时预览、工具栏、代码块、表格、链接、撤销重做等编辑能力。项目基于它做业务层扩展，比如多文档管理、本地保存、导入导出。

16. Vditor 工具栏是你自己写的吗？

不是，工具栏主要是 Vditor 自带的。项目通过配置告诉 Vditor 要显示哪些按钮，比如标题、加粗、斜体、链接、列表、表格、上传、全屏等。项目自己接管的是输入保存、图片上传处理、多文档切换和初始化内容。

17. 本地 Markdown 文件导入怎么实现？

导入用浏览器原生文件能力。点击导入按钮后，代码创建 input[type=file]，让用户选择本地 .md 文件。选中文件后，用 FileReader.readAsText 把文件读成字符串，再用文件名生成文档标题，新建文档、切换到这篇文档、保存内容，最后通知主编辑器刷新显示。

18. 为什么不能直接读取用户电脑上的文件？

浏览器出于安全限制，不允许网页随便读取本地文件。必须由用户通过 input[type=file] 主动选择某个文件，网页才有权限读取这个文件。FileReader 只能读取用户已经选择授权的文件。

19. 导出功能整体是怎么做的？

导出主要是先把 Markdown 渲染成 HTML 预览，再根据不同格式做转换。PNG 用 html2canvas 截 DOM，PDF 用 html2pdf.js 转 PDF，Word 用 HTML 转 DOCX，HTML 是组装独立 HTML 文件下载，PPT 用 Reveal.js 做浏览器里的幻灯片预览。

20. 为什么导出要先渲染成预览 DOM，而不是直接导出 Markdown 原文？

因为用户想导出的通常是看到的效果，不是 Markdown 源码。Markdown 原文只是文本，真正显示时会变成标题、加粗文本、表格、代码块等 HTML 结构。先渲染成预览 DOM，再导出，可以尽量做到所见即所得。

21. PPT 预览为什么用 Reveal.js？

Reveal.js 本身就是浏览器端幻灯片框架，而且支持 Markdown 插件。可以把当前 Markdown 内容按 --- 和 -- 分割成幻灯片，其中 --- 表示横向分页，-- 表示纵向分页。但它是浏览器预览，不是生成真正 .pptx 文件。

22. 移动端适配你做了什么？

移动端主要处理侧边栏和编辑空间问题。桌面端可以常驻侧边栏和分屏预览，但小屏幕空间有限，所以移动端侧边栏需要通过按钮展开和收起，避免挡住编辑区。同时移动端优先保证编辑区可用，而不是强行展示完整分屏预览。

23. 项目里做了哪些异常处理？

主要包括 localStorage 读取失败或数据异常时兜底为空数据，图片上传失败时提示用户，导出时如果预览 DOM 还没渲染完成会提示稍后再试，文件导入限制 Markdown 文件类型，删除文档前做确认，避免误删。

24. 如果 localStorage 容量不够怎么办？

localStorage 容量有限，只适合轻量本地数据。如果文档数量变多或内容变大，后续可以换成 IndexedDB。因为项目已经封装了存储适配层，所以从 localStorage 迁移到 IndexedDB 时，可以尽量保持 store 和组件逻辑不变，只替换底层适配器。

25. 如果要做云端同步，你会怎么扩展？

我会在现有存储适配层基础上扩展一个云端 Adapter。组件和 Pinia 仍然调用统一的文档操作方法，底层 Adapter 决定写 localStorage、IndexedDB 还是调用远程 API。同时需要增加用户登录、文档版本号、更新时间、同步状态和冲突处理。

26. 如果多个标签页同时编辑，会有什么问题？

可能会出现内容覆盖。比如两个标签页打开同一篇文档，A 保存后，B 还停留在旧内容。如果 B 再保存，就可能覆盖 A 的修改。可以监听 storage 事件或使用 BroadcastChannel，发现其他标签页修改当前文档时提示用户刷新或处理冲突。

27. 这个项目的核心难点是什么？

核心难点是数据一致性和浏览器端文件处理。数据一致性主要体现在多文档切换和自动保存，用户输入、切换、刷新、删除都可能影响当前文档状态，需要保证内容不丢、不串文档。文件处理主要体现在本地导入和多格式导出。

28. 如果让你总结这个项目的技术亮点，你会怎么说？

我会总结为四点：用 Vditor 接入 Markdown 编辑和预览能力；用 Pinia 统一管理文档状态；设计 localStorage 适配层，并把文档元数据和正文分离存储；实现浏览器端本地导入和多格式导出。

二、超短背诵稿

这个项目是一个纯前端 Markdown 文档工作台。编辑器用 Vditor，状态用 Pinia 的 docStore，本地持久化用 LocalStorageAdapter 封装 localStorage。页面打开时，Main.vue 通过 store.init 从 localStorage 恢复文档列表和当前正文，然后把内容 setValue 到 Vditor。用户输入时，Vditor 的 input 触发 Main.vue 的 1 秒防抖保存，最终通过 store.saveContent 写入 localStorage。左侧 Sidebar 只负责文档列表交互，点击文档时通知 Main.vue，由 Main.vue 先保存旧文档，再切换新文档并更新 Vditor。顶部 HeaderNav 负责导入、导出跳转和全屏。导入用 input[type=file] 和 FileReader 读取本地 md 文件，再创建新文档并通知 Main.vue 刷新。导出时，PNG/PDF/DOCX/HTML 先用 PreviewVditor 渲染预览 DOM，再用对应库转换；PPT 用 Reveal.js 做幻灯片预览。

三、核心链路速记

页面打开：localStorage -> docStore -> Main.vue -> Vditor

用户输入：Vditor -> Main.vue 防抖 -> docStore -> localStorage

切换文档：Sidebar -> Main.vue -> docStore -> Vditor

导入文件：HeaderNav -> input[type=file] -> FileReader -> createDoc/selectDoc/saveContent -> Main.vue 刷新

普通导出：当前 Markdown -> PreviewVditor -> 预览 DOM -> PNG/PDF/DOCX/HTML

PPT 预览：当前 Markdown -> ExportPPT -> Reveal.js -> 幻灯片预览